

Dashboard Estratégico — BeOn Lab

Sumário detalhado de métricas da aba Estratégia • Pipeline de Inovação • Experimentos • Lead Time

1. Resumo Executivo

1.1 Metas Estratégicas (EBITDA, Receita, NPS)

Cada Epic do board 2735 recebe uma classificação de meta via LLM (Azure OpenAI). O modelo analisa o summary (título) e o domínio do Epic para determinar se ele contribui para EBITDA, Receita ou NPS.

Fluxo de classificação:

1. No servidor (page.tsx), os Epics são extraídos do Jira via `fetchDashboardRaw()`
2. Para cada Epic, extrai-se `key + summary + domínio` -> envia para `classifyPortfolios()`
3. `classifyPortfolios()` chama a LLM (Azure OpenAI) que retorna { chave: "EBITDA"|"Receita"|"NPS"|null }
4. O resultado é armazenado em `portfolioClassification: Record<string, MetaCategoria>`
5. `buildDashboardData()` aplica essa classificação a cada `EpicDetail` via `metaCategoria`

Cálculo das Metas Agregadas (metasAgregadas):

```
// Algoritmo em mappers.ts (buildDashboardData)
const metasAgregadas = {
  EBITDA: {count:0, valor:0},
  NPS: {count:0, valor:0},
  Receita: {count:0, valor:0}
}

for each Iniciativa:
  seenMetas = set()
  for each Epic da Iniciativa:
    meta = epic.metaCategoria
    if meta AND meta NOT in seenMetas:
      seenMetas.add(meta)
      metasAgregadas[meta].count++
      metasAgregadas[meta].valor +=
        iniciativa.beneficioQuantitativoTotal

// Exibição no ResumoExecutivo.tsx:
totalMetasValor = EBITDA.valor + Receita.valor + NPS.valor
pct = (meta.valor / totalMetasValor) * 100
```

Detalhes importantes:

- count = número de Iniciativas (não de Epics) que possuem pelo menos um Epic com aquela meta
- valor = soma do `beneficioQuantitativoTotal` da Iniciativa (soma de todos os seus Epics)
- Uma Iniciativa pode contribuir para múltiplas metas se tiver Epics de categorias diferentes
- A barra de progresso mostra $pct = (valor\ da\ meta / total\ das\ 3\ metas) * 100$

1.2 Pipeline de Conversão

Benefício Potencial Total:

```
// Em buildDashboardData() - mappers.ts
epicsComBeneficio = allEpicDetails
  .filter(e => (e.beneficioQuantitativo ?? 0) > 0)
beneficioTotal = epicsComBeneficio
  .reduce((s, e) => s + (e.beneficioQuantitativo ?? 0), 0)
beneficioMedio = epicsComBeneficio.length > 0
  ? beneficioTotal / epicsComBeneficio.length
  : 0

// Fonte: customfield_30216 (campo "Benefício Quantitativo")
// Preenchido manualmente pelo usuário ao cadastrar o experimento
```

Conversão para Escala:

```
// No ResumoExecutivo.tsx
totalExperimentos = allEpics
  .filter(e => e.status.id !== '10015').length
// Exclui apenas cancelados (10015)
emEscala = pipeline['EM ESCALA']
taxaEscala = totalExperimentos > 0
  ? Math.round((emEscala / totalExperimentos) * 100)
  : 0

// Exemplo: 3 em escala de 199 totais -> taxaEscala = 1.5%
```

Conversao para Piloto:

```
// No ResumoExecutivo.tsx
emPilotoEscala = pipeline['EM PILOTO']
+ pipeline['EM ESCALA']
taxaPiloto = totalExperimentos > 0
? Math.round((emPilotoEscala / totalExperimentos) * 100)
: 0
```

2. Portfolio por Mercado

Cada Epic e classificado em um dos 3 segmentos de mercado via LLM (Azure OpenAI), analisando o campo dominio do Epic.

Fluxo de classificacao de segmento:

1. Extrai-se key + summary + dominio de cada Epic -> envia para classifySegmentos()
2. A LLM retorna { chave: "Consumo"|"Corporativo"|"PME/GE/GOV" }
3. Resultado armazenado em segmentoClassification: Record<string, SegmentoMercado>
4. Fallback: dominio "Empresarial" ou "PME" -> "PME/GE/GOV", senao "Consumo"

Calculo dos agregados por segmento (mercadosSegmento):

```
// Em buildDashboardData() - mappers.ts
SEGMENTOS = ['Consumo', 'Corporativo', 'PME/GE/GOV']
segMap = new Map(SEGMENTOS.map(s => [s, []]))

for each EpicDetail e:
  seg = segmentoClassification[e.key]
  ?? getSegmentoFallback(dominioByKey.get(e.key))
  segMap.get(seg).push(e)

for each segmento:
  qtdExperimentos = epics.length
  valorPotencial = SUM(e.beneficioQuantitativo ?? 0)
  dominios = top 4 dominios mais frequentes
  pct = (count / total) * 100
  alertas:
    bloqueadosIA = epics with motivoBloqueio
    aguardandoDelivery = epics status 10067
    semSponsor = epics without sponsor
```

3. Funil de Experimentos

O funil combina dados dos dois boards Jira. As camadas 1 a 5 usam o board 2735 (Epics/experimentos). As camadas 6 e 7 usam o board 2734 (Iniciativas) para medir a transicao para Piloto e Escala.

Calculo detalhado de cada camada:

```
// No FunilExperimentos.tsx - board 2735 (Epics)
totalExperimentos = allEpics.length
cancelados = allEpics.filter(e => e.status.id === '10015').length
emAndamento = allEpics.filter(e => e.status.id === '3').length
emValidacao = allEpics.filter(e => e.status.id === '10204').length
concluidos = allEpics.filter(e =>
  e.status.id === '10003' || e.status.id === '10019').length

// Board 2734 (Iniciativas) - pipeline counts
pilotos = pipeline['EM PILOTO']
escala = pipeline['EM ESCALA']

// Taxas de conversao:
taxaCancelamento = (cancelados / total) * 100
taxaAndamento = (emAndamento / total) * 100
taxaValidacao = (emValidacao / emAndamento) * 100
taxaConclusao = (concluidos / emValidacao) * 100
taxaPiloto = (pilotos / concluidos) * 100
taxaEscala = (escala / pilotos) * 100
```

Exemplo numerico:

Com 199 experimentos totais, 20 cancelados, 50 em andamento, 8 em validacao, 15 concluidos, 3 pilotos, 1 escala:

- Cancelamento: $20/199 = 10\%$ | Andamento: $50/199 = 25\%$ | Validacao: $8/50 = 16\%$
- Conclusao: $15/8 = 187\%$ (pode superar 100% se concluidos incluem itens que pularam validacao)
- Piloto: $3/15 = 20\%$ | Escala: $1/3 = 33\%$

4. Top 5 Experimentos

Selecione os 5 experimentos com maior benefício quantitativo estimado (R\$).

```
// Em buildDashboardData() - mappers.ts
top5Epics = allEpicDetails
  .filter(e => (e.beneficioQuantitativo ?? 0) > 0)
  .sort((a, b) =>
    (b.beneficioQuantitativo ?? 0)
    - (a.beneficioQuantitativo ?? 0))
  .slice(0, 5)

// Ordenacao secundaria: por prioridade
// PRIORITY_ORDER = ['High', 'Medium', 'Low']
```

Colunas exibidas no componente Top5Experimentos.tsx:

- Experimento (nome + key)
- Lab Responsavel (customfield_31438 ou customfield_30357)
- Dominio (customfield_30021 ou fallbacks)
- Valor Potencial (beneficioQuantitativo formatado em R\$)

5. Crescimento da Experimentacao (Burnup)

O burnup mostra o acumulado mes a mes de experimentos concluidos e seu beneficio total. E calculado por buildMonitoramentoData() em mappers.ts.

```
// Algoritmo simplificado de buildMonitoramentoData()
// Para cada mes desde a data mais antiga ate hoje:
for each mes:
  realizado[mes] = COUNT de Epics com status 10003
  E concluidoEm <= fim do mes
  beneficio[mes] = SUM(beneficioQuantitativo)
  desses Epics

// Acumulado: cada mes soma o valor do mes anterior
// Ex: Jan=2, Fev=3, Mar=1 -> acum: Jan=2, Fev=5, Mar=6

// Metricas derivadas:
experimentosConcluidos = total Epics status 10003
taxaConversao = experimentosConcluidos
  / total experimentos * 100
```

A data de conclusao (concluidoEm) e extraida do changelog:

- Varre o changelog do Epic em ordem cronologica
- Encontra a primeira entrada onde status mudou para "Concluido" (10003) ou "FINALIZADO" (10019)
- Usa a data (created) dessa entrada como concluidoEm

6. Jornada de Adocao (Lead Time)

6.1 Timeline - 4 Fases

A Jornada de Adocao mede o tempo medio (em dias) que um experimento leva para percorrer cada fase do pipeline. O Backlog e excluido do total - a jornada começa na Experimentacao.

Fase 1 - Experimentacao:

```
// calculateCycleTimeExperimentacaoDetalhado()
// Apenas Epics CONCLUIDOS (status 10019)
EXPERIMENTACAO_NAMES = {
  'Em andamento', 'In Progress', 'EM VALIDACAO'
}

for each Epic concluido:
  changelog = epicChangelogs[epic.key]
  sorted = changelog.sort(created ASC)

  // Encontra periodos em experimentacao
  periodos = []
  entrouEm = null
  for each entry in sorted:
    for each item in entry.items:
      if item.field === 'status':
        if item.toString in EXPERIMENTACAO_NAMES
          AND item.fromString NOT in:
            entrouEm = entry.created
        if item.fromString in EXPERIMENTACAO_NAMES
          AND item.toString NOT in:
```

Fase 2 - Transicao para Piloto:

```
// Media de dias das Iniciativas no status
// AGUARDANDO PILOTO (board 2734)
// Usa changelog das Iniciativas
```

Fase 3 - Piloto:

```
// Media de dias das Iniciativas no status
// EM PILOTO (board 2734)
```

Fase 4 - Escala:

```
// Media de dias das Iniciativas no status
// EM ESCALA (board 2734)
// Se nao houver dados, a fase e omitida
```

Agregacao final (calculateLeadTimeJornada):

```
// No mappers.ts
totalDias = experimentacaoDias
+ transicaoPilotoDias
+ pilotoDias
+ (escalaDias > 0 ? escalaDias : 0)

pct = (diasFase / totalDias) * 100

// Bottleneck: fase com maior numero de dias
bottleneck = fases.sort(desc por dias)[0]

// Decomposicao:
tempoGeracaoValorDias = experimentacaoDias + pilotoDias
tempoEsperaTransicaoDias = backlogDias + transicaoPilotoDias
tempoImplantacaoEscalaDias = escalaDias
```

6.2 Cycle Time por Complexidade

O cycle time de experimentacao e quebrado por porte (complexidade do Epic), usando o campo customfield_30358.

```
// calculateCycleTimeExperimentacaoDetalhado()
PORTE_MAP = {
  'Baixa': 'P', 'Media': 'M', 'Alta': 'G',
  'P': 'P', 'M': 'M', 'G': 'G'
}

for each Epic concluido:
  cycleTime = calcularCycleTime(epic)
  complexidadeRaw = epic.fields.customfield_30358
  porte = PORTE_MAP[complexidadeRaw] ?? 'Sem porte'
  porteCycleTimes[porte].push(cycleTime)

// Para cada porte:
media = SUM(dias) / count
mediana = valor do meio do array ordenado
blockedTimeMedio = SUM(dias bloqueados) / count

// Ordem: P (Baixa) -> M (Media) -> G (Alta)
```

6.3 Blocked Time

O Blocked Time mede quantos dias, em media, os experimentos concluidos ficaram bloqueados (com motivo de bloqueio preenchido) durante seu ciclo de vida.

```
// calculateLeadTimeJornada() - mappers.ts
// Apenas Epics CONCLUIDOS (status 10019)
for each Epic concluido:
  changelog = epicChangelogs[epic.key]
  sorted = changelog.sort(created ASC)

  // Encontra data de conclusao
  fimCiclo = Date.now() // fallback
  for each entry in sorted:
    for each item in entry.items:
      if item.field === 'status'
        AND item.toString === '10019':
          fimCiclo = entry.created

  totalDias = (fimCiclo - criadoEm)
  / (1000*60*60*24)
  lifecycleTotals.push(totalDias)
```

Exemplo:

Se um Epic levou 90 dias no total (criacao -> conclusao) e ficou bloqueado por 15 dias:
- blockedTimeDias = 15 | blockedTimePct = 15/90 = 16.7%
- Isso significa que ~17% do ciclo de vida foi desperdicado em bloqueios

7. Filtro por Periodo

O filtro de periodo (Header) refaz todos os calculos do dashboard filtrando os dados pela data de criacao. A funcao filtrarDashboardData() em EstrategiaClient.tsx e responsavel por isso.

```
// Em EstrategiaClient.tsx - filtrarDashboardData()
// 1. Filtra iniciativas por criadoEm
iniciativasFiltradas = iniciativas
    .filter(ini => isDataNoPeriodo(ini.criadoEm, periodo))

// 2. Para cada iniciativa, filtra seus Epics
ini.epics = ini.epics
    .filter(epic => isDataNoPeriodo(epic.criadoEm, periodo))

// 3. Recalcula beneficioQuantitativoTotal
ini.beneficioQuantitativoTotal = SUM(
    epicsFiltrados.beneficioQuantitativo)

// 4. Reagrega dominios, sponsors, segmentos
ini.dominios = Array.from(new Set(
    epicsFiltrados.map(e => e.dominio)))
ini.sponsors = Array.from(new Set(
    epicsFiltrados.map(e => e.sponsor)))

// 5. Recalcula pipeline (contagem por coluna)
// 6. Recalcula beneficioTotal, beneficioMedio
// 7. Recalcula mercados, mercadosSegmento,
//    top5Epics, topSponsors
// 8. Recalcula statusDistribuicao (donut),
//    metasAgregadas, iniciativasPorMeta

// Opcoes de periodo:
// 'tudo' | 'ultimos12' | 'ultimos6'
// | 'ultimos3' | 'ultimoMes' | personalizado
```

8. Origem dos Dados

Board	ID Jira	Issue Type	Funcao	Qtd Aprox.
P&D - Ideacao	2734	Iniciativa	Pipeline de inovacao (Backlog -> Escala)	~186
P&D - Experimentacao/Piloto	2735	Epic	Experimentos com beneficio, complexidade, sponsor	~199

Campos customizados do Jira utilizados:

Campo	ID	Uso
Beneficio Quantitativo	customfield_30216	Valor em R\$ do experimento
Complexidade	customfield_30358	Porte P/M/G para cycle time
Sponsor	customfield_30394	Patrocinador do experimento
Lab Responsavel	customfield_31438 / 30357	Time que executa o experimento
Dominio	customfield_30021 / 11987 / 11991	Area de negocio
Motivo de Bloqueio	customfield_13406	Causa do bloqueio (para blocked time)
Segmento	customfield_30445	Segmento de mercado
Portfolio	customfield_30110	Classificacao de portfolio

Classificacao LLM:

- Portfolio (EBITDA/Receita/NPS): Azure OpenAI - analisa summary + dominio de cada Epic
- Segmento (Consumo/Corporativo/PME): Azure OpenAI - analisa summary + dominio de cada Epic
- Cache: Resultados armazenados em .portfolio-cache.json e .segmento-cache-v2.json